

Motivation & Project Overview

Why Pathfinding?

Finding efficient routes through costly, uneven terrain is a core problem in robotics, logistics, and games. Comparing classical search strategies on realistic, weighted maps reveals trade-offs between optimality, speed, and memory that are invisible on simple, uniform-cost grids.

Project Overview

- Procedurally generate 2D terrain grids with elevation, obstacles, and configurable terrain types (plain, forest, mountain, swamp) - see pathfinding.ipynb.
- Model movement as a weighted graph: each move costs $1 + |\text{elevation difference}|$ between adjacent cells, and obstacle cells are removed from the graph entirely.
- Run five classical search algorithms - BFS, DFS, Uniform-Cost Search, Greedy Best-First, and A* - on three test scenarios: flat terrain, a mountain barrier, and random terrain.
- Evaluate every algorithm on path cost, path length, nodes explored, and runtime to compare optimality against computational efficiency.

Methods & Technologies

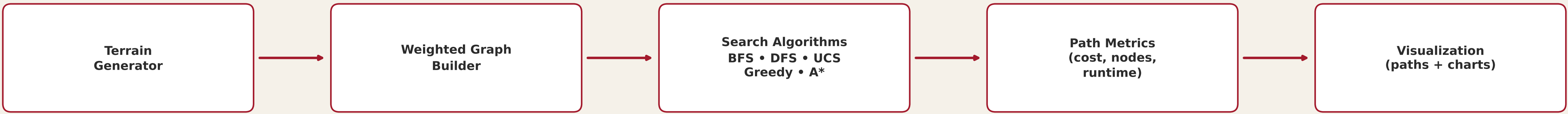
Key Methods

- Terrain representation - NumPy 2D elevation arrays + boolean obstacle masks (terrain.py: flat, mountain-barrier, and random scenarios).
- Graph construction - 4-connected grid graph; edge weight = $1 + \text{elevation change}$; obstacle cells excluded from the graph (graph.py).
- Uninformed search - Breadth-First Search and Depth-First Search (search.py).
- Cost-sensitive search - Uniform-Cost Search using a priority queue over path cost.
- Informed search - Greedy Best-First Search and A*, both guided by the Manhattan-distance heuristic to the goal.
- Evaluation - path length, weighted path cost, elevation gain, nodes explored, and wall-clock runtime, computed per run (metrics.py).

Technology Stack

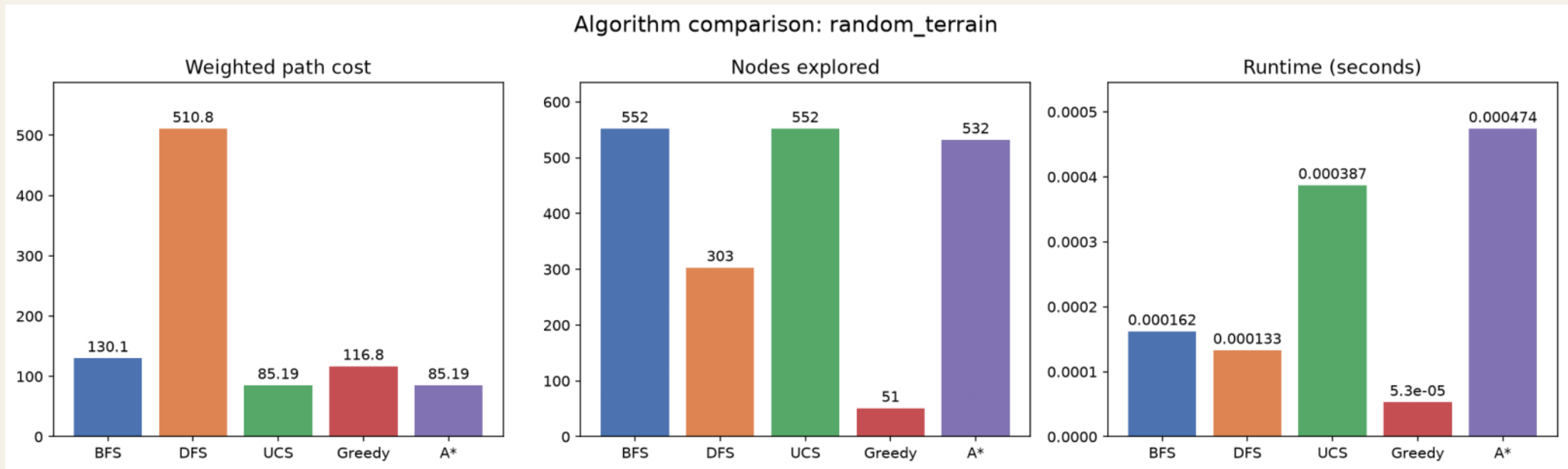
Python 3, NumPy for terrain/graph data, Matplotlib for all visualizations, heapq-based priority queues for UCS/Greedy/A*.

System / AI Pipeline



Results — Random Terrain

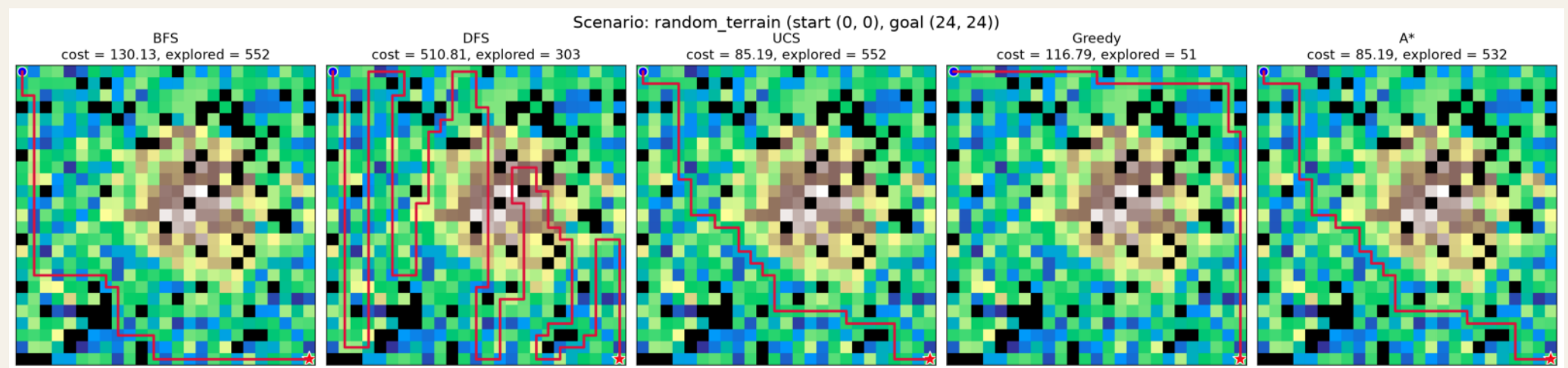
25×25 random-elevation map with scattered obstacles, start (0,0), goal (24,24). Weighted path cost and nodes explored for all five algorithms:



Algorithm	Path Cost	Explored	Runtime (s)
BFS	130.13	552	0.000162
DFS	510.81	303	0.000133
UCS	85.19	552	0.000387
Greedy	116.79	51	0.000053
A*	85.19	532	0.000474

UCS and A* both find the true minimum-cost path (85.19); A* reaches it while exploring roughly as many nodes as UCS but is guided toward the goal by its heuristic.

Visualization — Paths Found



Each panel shows the terrain (dark = low elevation, bright = high elevation, black = obstacle), the start (circle) and goal (star), and the explored path (red line) for one algorithm.

- BFS / UCS explore the entire reachable frontier; UCS returns the cheapest route.
- DFS commits to one direction and backtracks, producing a long, zig-zagging path.
- Greedy heads straight for the goal, exploring only 51 nodes but at a higher cost.
- A* combines UCS's cost-awareness with Greedy's directionality: same optimal cost as UCS, in a smaller, more targeted search.

Scenarios Tested

Every algorithm was run on three 25×25 scenarios with different elevation profiles to stress-test cost-awareness:

Scenario	UCS/A* Cost	BFS Cost	Greedy Cost	DFS Cost
Flat terrain	48.0	48.0	48.0	336.0
Mountain barrier	65.2	70.24	72.88	621.84
Random terrain	85.19	130.13	116.79	510.81

- Flat terrain: every algorithm ties on cost since all moves cost 1; DFS is still far worse because it does not search shortest-path-first.
- Mountain barrier: a costly elevation ridge separates start and goal; UCS/A* route around the cheapest crossing, while BFS's fewest-steps path pays a higher elevation penalty.
- Random terrain: scattered obstacles plus a smooth hill create the largest cost gap between informed (UCS/A*) and uninformed (BFS/DFS) search.

Key Findings & Discussion

- A* offers the best overall balance: it matches UCS's optimal path cost on every scenario while its heuristic keeps the explored state count competitive.
- Uniform-Cost Search is guaranteed optimal but explores the same large frontier as BFS - correct, but not efficient.
- Greedy Best-First is by far the fastest and explores the fewest nodes (e.g. 51 vs. 552 on random terrain), but ignores accumulated cost, so it becomes increasingly suboptimal as terrain gets more uneven.
- Breadth-First Search only finds the optimal path when the terrain is uniform-cost (flat terrain); once elevation varies, its fewest-edges path is no longer the cheapest one.
- Depth-First Search is unsuitable for cost-sensitive navigation in every scenario tested: it produced the longest, most expensive path each time, despite sometimes exploring few nodes.
- Takeaway: heuristic-guided, cost-aware search (A*) is the clear choice when both path quality and search efficiency matter.

References

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson. — BFS, DFS, Uniform-Cost Search, Greedy Best-First Search, and A* Search foundations.
- Project repository: github.com/jameszhan02/CMPT310_Path_Finding — pathfinding.ipynb (terrain generation/visualization) and cmpt310_milestone1_pathfinders/ (search algorithms, graph, metrics, scenarios).
- CMPT 310 - D100, Introduction to Artificial Intelligence, Lecture Notes. Simon Fraser University, School of Computing Science, Summer 2026.
- Python, NumPy, and Matplotlib documentation — used for all data structures and figures shown on this poster.